



Controles de selección

Una vez vistos los controles básicos que hay disponibles en Android, vamos a ver los controles de selección: Spinner (listas despegables) y RecyclerView (listas fijas y tablas). Todos los controles de selección tienen en común el uso de adaptadores.

Adaptadores (adapters)

Un adaptador actúa como un puente entre el AdapterView y los datos. Es decir, los controles de selección accederán a los datos a través de un adaptador.

Además, el adaptador será responsable de generar a partir de estos datos las vistas específicas que se mostrarán dentro del control de selección. Por ejemplo, si cada elemento de una lista estuviera formado por varias etiquetas y una imagen, los adaptadores serían los responsables de generar y establecer el contenido de estos subelementos.

Android nos proporciona una serie de adaptadores sencillos, aunque podemos extender su funcionalidad y adaptarlos fácilmente a nuestras necesidades específicas.

- **ArrayAdapter**. El más sencillo, y provee de datos a un control de selección a partir de un array de objetos de cualquier tipo.
- **SimpleAdapter**. Se utiliza para mapear datos sobre los diferentes controles definidos en un fichero XML de layout.
- **SimpleCursorAdapter**. Se utiliza para mapear las columnas de un cursor abierto sobre una base de datos sobre los diferentes elementos visuales contenidos en el control de selección.

Para crear un adaptador de tipo ArrayAdapter

```
val adaptador = ArrayAdapter(this, android.R.layout.simple_spinner, datos)
```

Donde el adaptador recibe el contexto, el ID del layout (en este caso es un predefinido) y un array que hemos llamado datos.



Spinner

Las listas desplegables en Android se llaman Spinner. el usuario selecciona la lista, se muestra una especie de lista emergente al usuario con todas las opciones disponibles y al seleccionarse una de ellas ésta queda fijada en el control.

```
<Spinner
    android:id="@+id/spinner"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
/>
```

Para enlazar nuestro adaptador con los datos a este control utilizaremos el siguiente código:

```
val datos = arrayOf("Elemento1", "Elemento2", "Elemento3",
"Elemento4", "Elemento5")
val adaptador = ArrayAdapter(this,
android.R.layout.simple_spinner_item, datos)
adaptador.setDropDownViewResource (
    android.R.layout.simple_spinner_dropdown_item)
spinner.adapter = adaptador
```

El método `setDropDownViewResource(ID_layout)` define el diseño (layout) específico que se utilizará para mostrar los elementos dentro de la lista emergente (el dropdown).

Es crucial usar dos layouts distintos:

- El primer layout (`android.R.layout.simple_spinner_item`) define la apariencia del elemento seleccionado y visible en el Spinner principal.
- El segundo layout (`android.R.layout.simple_spinner_dropdown_item`) define la apariencia de los elementos dentro del menú desplegable, que suele tener un mayor padding (relleno) para una mejor experiencia de selección.

Ambos layouts predefinidos (tanto `simple_spinner_item` como `simple_spinner_dropdown_item`) están formados internamente por una única vista de texto (TextView).

Otra opción es definir una lista de valores posibles como un recurso en la carpeta `/res/values`



```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <string-array name="valores_array">
        <item>Elem1</item>
        <item>Elem2</item>
        <item>Elem3</item>
        <item>Elem4</item>
        <item>Elem5</item>
    </string-array>
</resources>
```

y modificar la creación del adaptador de la siguiente forma:

```
val adaptador=ArrayAdapter.createFromResource(this,
R.array.valores_array, android.R.layout.simple_spinner_item)
//val adaptador = ArrayAdapter(this,
android.R.layout.simple_spinner_item, datos)
```

En cuanto a los eventos lanzados por el control Spinner, el más común es el generado al seleccionarse una opción de la lista desplegable, *onItemSelected*. Para capturar este evento se procederá de forma similar a lo ya visto para otros controles anteriormente, asignándole su controlador mediante la propiedad *onItemSelectedListener*:

```
spinner.onItemSelectedListener = object:
    AdapterView.OnItemSelectedListener {

        override fun onItemSelected(parent: AdapterView<*>, view:
View?, position: Int, id: Long) {
            val pos = parent.getItemAtPosition(position)

            Toast.makeText(this@MainActivity, "se ha seleccionado
$pos", Toast.LENGTH_SHORT).show()
        }

        override fun onNothingSelected(p0: AdapterView<*>?) {
            TODO("Not yet implemented")
        }
    }
}
```



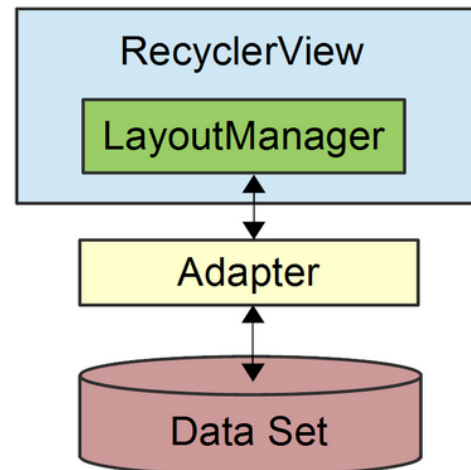
RecyclerView

Control de tipo lista o tabla. Crea listas deslizables formadas a partir de otras vistas. Sustituye a ListView y GridView

RecyclerView necesita de los siguientes componentes complementarios.

- RecyclerView.Adapter
- RecyclerView.ViewHolder
- LayoutManager
- ItemDecoration
- ItemAnimator

El view holder se encargará de contener y gestionar las vistas o controles asociados a cada elemento individual de la lista. El control RecyclerView se encargará de crear tantos view holder como sea necesario para mostrar los elementos de la lista que se ven en pantalla y los gestionará eficientemente de forma que no tenga que crear nuevos objetos para mostrar más elementos de la lista al hacer scroll, sino que tratará de «reciclar» aquellos que ya no sirven por estar asociados a otros elementos de la lista que ya han salido de la pantalla. También organiza los eventos de click que se reenvía al adaptador.

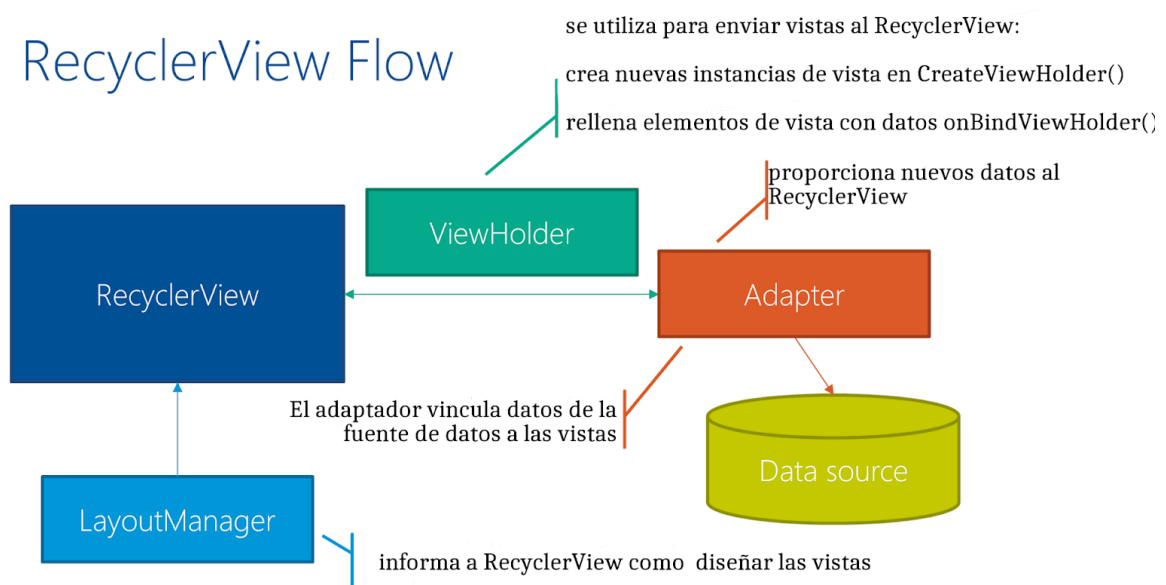


Una vista de tipo RecyclerView por el contrario no determina por sí sola la forma en que se van a mostrar en pantalla los elementos de nuestra colección, sino que va a delegar esa tarea a otro componente llamado LayoutManager, que también tendremos que crear y asociar al RecyclerView para su correcto funcionamiento. El SDK incorpora una serie de LayoutManagers para las representaciones mas habituales (LinearLayoutManager, GridLayoutManager, StaggeredGridLayoutManager).

RecyclerView.Adapter: proporciona un enlace desde el conjunto de datos de la aplicación a las vistas de elementos que se muestran dentro de RecyclerView. El adaptador sabe cómo asociar cada posición de vista de elemento en RecyclerView a una ubicación específica del origen de datos

Los dos últimos componentes de la lista se encargarán de definir cómo se representarán algunos aspectos visuales concretos de nuestra colección de datos

RecyclerView Flow



El uso de RecyclerView requiere configurar/implementar los siguientes componentes:

- **RecyclerView:** gestiona todo. Está en su mayor parte escrito por Android. Tenemos que proporcionar los componentes y la configuración.
- **Adaptador:** esta es la clase que más hay que codificar. Se conecta a la fuente de datos. Cuando RecyclerView lo indica, crea o actualiza los elementos individuales de la lista.
- **ViewHolder:** una clase simple que asigna/actualiza los datos en los elementos de la vista. Cuando se reutiliza una vista, los datos anteriores se sobrescriben.
- **Fuente de datos:** cualquier cosa que desee, desde una simple matriz hasta una fuente de datos completa. Su Adaptador interactúa con él.
- **LayoutManager:** es responsable de colocar todos los elementos de vista individuales en la pantalla y asegurarse de que obtengan el espacio de pantalla que necesitan.

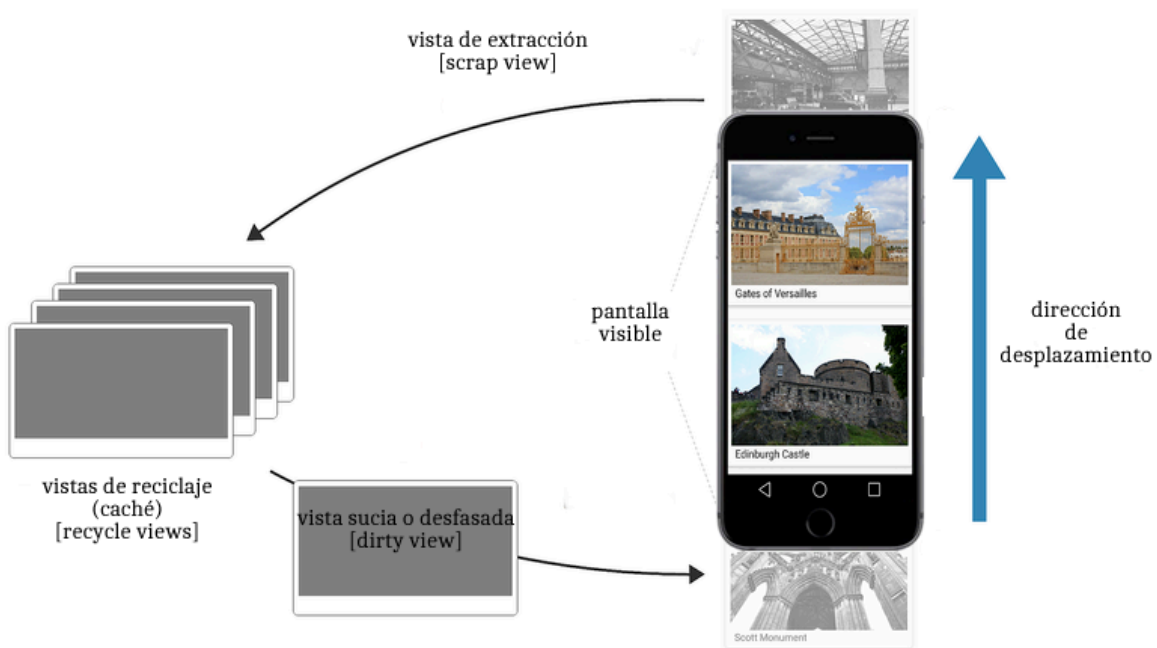
El Adaptador hace la mayor parte del trabajo para RecyclerView : conecta la fuente de datos para ver elementos.

1. Para dibujar la lista en la pantalla, RecyclerView le preguntará al Adaptador cuántos elementos hay en total. Nuestro adaptador devuelve esta información en `getItemCount`

2. Siempre que RecyclerView decida que necesita otra instancia de vista en la memoria, llamará onCreateViewHolder (). En este método, el Adaptador se infla y devuelve el diseño xml
3. Cada vez que se (re)utiliza un ViewHolder creado previamente , RecyclerView le indica al Adaptador que actualice sus datos. Personalizamos este proceso sobreecribiendo onBindViewHolder ()

¿Cómo funciona?

RecyclerView no asigna una vista para cada elemento del origen de datos, si no que solo asigna el número de vistas que caben en la pantalla y reutiliza esos diseños a medida que el usuario se va desplazando .



Agregar un RecyclerView a un diseño es simplemente una cuestión de agregar el elemento apropiado al archivo de diseño de contenido XML de la actividad en la que aparecerá. Por ejemplo:

```
<androidx.recyclerview.widget.RecyclerView
```



```
android:id="@+id/recyclerViewPelículas"  
android:layout_width="match_parent"  
android:layout_height="match_parent"  
android:layout_marginHorizontal="8dp"  
android:layout_marginVertical="16dp" />
```

[**Ver ejemplo recyclerview](#)

Oyentes del RecyclerView

método 1

El funcionamiento de los oyentes es igual a los visto anteriormente. En este caso podemos poner en nuestro ViewHolder personalizado el código correspondiente.

```
itemView.setOnClickListener{  
    Log.d("listener", "viewholderlistener "+this.layoutPosition)  
}
```

método 2

dentro del método onBindViewHolder

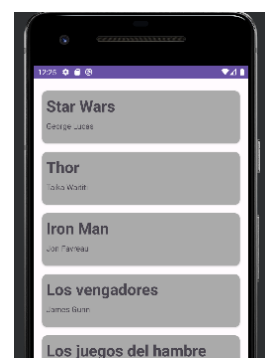
```
holder.itemView.setOnClickListener {  
    Log.d("recycler", peli.director);  
}
```

método 3

con callbacks, que se estudiarán más adelante

CardView

La clase CardView es una vista de interfaz de usuario que permite presentar información en grupos utilizando una metáfora de tarjeta. Las tarjetas se presentan normalmente en listas utilizando una instancia de





RecyclerView y pueden configurarse para que aparezcan con efectos de sombra y esquinas redondeadas

Para controlar el nivel de sombras se usa el atributo `card_view:cardElevation` y para los bordes el atributo `card_view:cardCornerRadius`.